

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
!pip install -q grad-cam datasets
```

```
7.8/7.8 MB 105.9 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for grad-cam (pyproject.toml) ... done
```

```
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn

from torchvision import models
from torchvision import transforms

from torch.utils.data import Dataset
from torch.utils.data import DataLoader

from datasets import load_dataset

from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
```

```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print(device)
```

cuda

```
dataset = load_dataset(
    "electricsheepafrica/Africa-Blood-Cell-Images-and-EHR-for-Cancer-Detection"
)
```

```
README.md: 100% 7.24k/7.24k [00:00<00:00, 621kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
dataset_infos.json: 100% 3.69k/3.69k [00:00<00:00, 310kB/s]
data/train-00000-of-00001.parquet: 100% 129M/129M [00:05<00:00, 31.7MB/s]
Generating train split: 100% 5000/5000 [00:00<00:00, 7492.64 examples/s]
```

```
split_dataset = dataset["train"].train_test_split(
    test_size=0.2,
    seed=42
)

train_data = split_dataset["train"]
test_data = split_dataset["test"]
```

```
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])
```

```
class BloodDataset(Dataset):

    def __init__(self, hf_dataset, transform=None):
```

```

        self.dataset = hf_dataset
        self.transform = transform

    def __len__(self):

        return len(self.dataset)

    def __getitem__(self, idx):

        item = self.dataset[idx]

        image = item["image"]

        label = item["label"]

        if self.transform:
            image = self.transform(image)

        return image, label

```

```

train_dataset = BloodDataset(
    train_data,
    transform
)

test_dataset = BloodDataset(
    test_data,
    transform
)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)

```

```

num_classes = 5

model = models.resnet18(
    weights=models.ResNet18_Weights.DEFAULT
)

model.fc = nn.Sequential(
    nn.Dropout(0.3),
    nn.Linear(
        model.fc.in_features,
        num_classes
    )
)

model = model.to(device)

```

Downloading: "<https://download.pytorch.org/models/resnet18-f37072fd.pth>" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth [100%|██████████| 44.7M/44.7M [00:00<00:00, 141MB/s]]

```

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)

```

```

num_epochs = 10

history = {
    "train_loss": [],
    "train_acc": []
}

for epoch in range(num_epochs):

    model.train()

```

```

running_loss = 0
correct = 0
total = 0

for images, labels in train_loader:

    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad()

    outputs = model(images)

    loss = criterion(
        outputs,
        labels
    )

    loss.backward()

    optimizer.step()

    running_loss += loss.item()

    preds = outputs.argmax(1)

    correct += (
        preds == labels
    ).sum().item()

    total += labels.size(0)

epoch_loss = (
    running_loss / len(train_loader)
)

epoch_acc = correct / total

history["train_loss"].append(
    epoch_loss
)

history["train_acc"].append(
    epoch_acc
)

print(
    f"Epoch [{epoch+1}/{num_epochs}] "
    f"Loss={epoch_loss:.4f} "
    f"Acc={epoch_acc:.4f}"
)

```

```

Epoch [1/10] Loss=0.1319 Acc=0.9613
Epoch [2/10] Loss=0.0191 Acc=0.9942
Epoch [3/10] Loss=0.0104 Acc=0.9975
Epoch [4/10] Loss=0.0043 Acc=0.9992
Epoch [5/10] Loss=0.0022 Acc=0.9995
Epoch [6/10] Loss=0.0055 Acc=0.9980
Epoch [7/10] Loss=0.0121 Acc=0.9960
Epoch [8/10] Loss=0.0018 Acc=0.9998
Epoch [9/10] Loss=0.0051 Acc=0.9990
Epoch [10/10] Loss=0.0023 Acc=0.9990

```

```

model.eval()

all_preds = []
all_labels = []

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)

        outputs = model(images)

        preds = outputs.argmax(1)

        all_preds.extend(
            preds.cpu().numpy()
        )

        all_labels.extend(

```

```
        labels.numpy()
    )
```

```
def enable_dropout(model):

    for m in model.modules():

        if isinstance(
            m,
            nn.Dropout
        ):
            m.train()
```

```
class_names = [
    'basophil',
    'erythroblast',
    'monocyte',
    'myeloblast',
    'seg_neutrophil'
]
```

```
torch.save(
    model.state_dict(),
    "/content/drive/MyDrive/trushworthy_ai/best_model.pth"
)
```

```
model.eval()
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
)
```

```
(relu): ReLU(inplace=True)
```

```
from sklearn.metrics import confusion_matrix
```

```
target_layers = [  
    model.layer4[-1]  
]
```

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score  
)  
  
acc = accuracy_score(  
    all_labels,  
    all_preds  
)  
  
precision = precision_score(  
    all_labels,  
    all_preds,  
    average="macro"  
)  
  
recall = recall_score(  
    all_labels,  
    all_preds,  
    average="macro"  
)  
  
f1 = f1_score(  
    all_labels,  
    all_preds,  
    average="macro"  
)  
  
print("="*50)  
print(f"Accuracy : {acc:.4f}")  
print(f"Precision: {precision:.4f}")  
print(f"Recall   : {recall:.4f}")  
print(f"F1 Score  : {f1:.4f}")  
print("="*50)
```

```
=====  
Accuracy : 0.9980  
Precision: 0.9980  
Recall   : 0.9980  
F1 Score : 0.9980  
=====
```

```
from sklearn.metrics import classification_report
```

```
print(  
    classification_report(  
        all_labels,  
        all_preds,  
        target_names=class_names,  
        digits=4  
    )  
)
```

	precision	recall	f1-score	support
basophil	1.0000	0.9949	0.9974	196
erythroblast	1.0000	1.0000	1.0000	197
monocyte	0.9950	1.0000	0.9975	200
myeloblast	0.9952	1.0000	0.9976	208
seg_neutrophil	1.0000	0.9950	0.9975	199
accuracy			0.9980	1000
macro avg	0.9980	0.9980	0.9980	1000
weighted avg	0.9980	0.9980	0.9980	1000

```
from sklearn.metrics import confusion_matrix  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
cm = confusion_matrix(  
    all_labels,
```

```

    all_preds
)

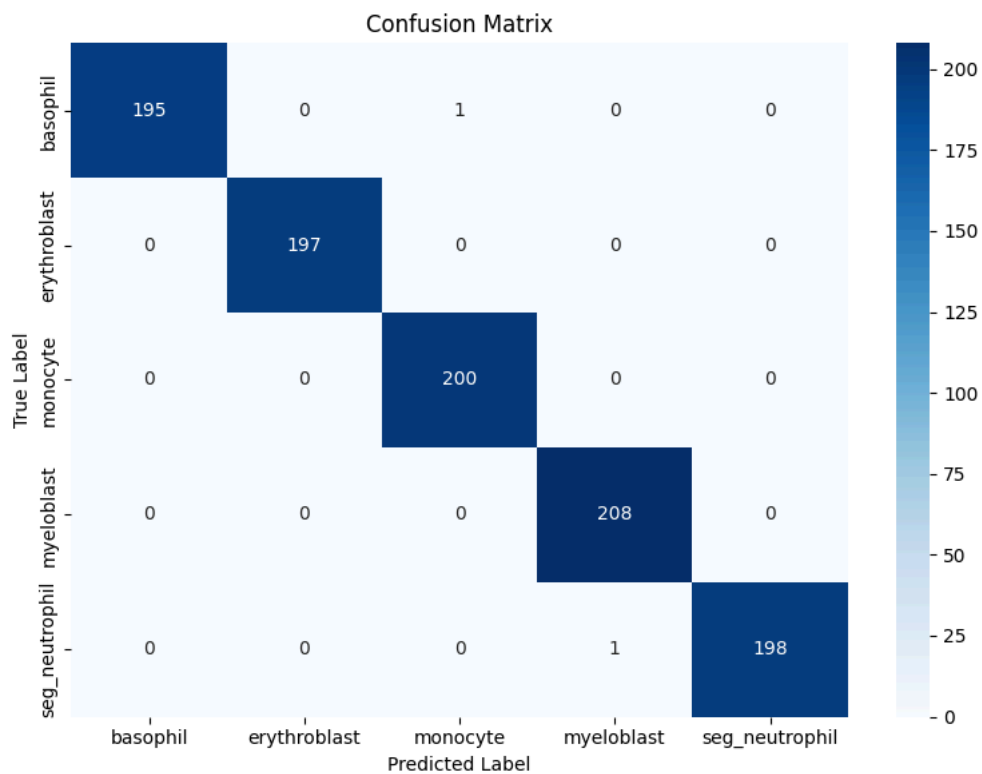
plt.figure(figsize=(8,6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")

plt.tight_layout()
plt.show()

```



```

import torch.nn.functional as F

model.eval()
target_layers = [
    model.layer4[-1]
]

cam = GradCAM(
    model=model,
    target_layers=target_layers
)

for i in range(5):

    image, label = test_dataset[i]

    input_tensor = (
        image.unsqueeze(0)
        .to(device)
    )

    with torch.no_grad():

        output = model(
            input_tensor
        )

```

```

        probs = F.softmax(
            output,
            dim=1
        )

        pred = probs.argmax(1).item()

        confidence = probs.max().item()

    grayscale_cam = cam(
        input_tensor=input_tensor
    )[0]

    rgb_img = image.permute(
        1,2,0
    ).cpu().numpy()

    rgb_img = (
        rgb_img-rgb_img.min()
    ) / (
        rgb_img.max()-rgb_img.min()
    )

    cam_image = show_cam_on_image(
        rgb_img,
        grayscale_cam,
        use_rgb=True
    )

    plt.figure(figsize=(10,4))

    plt.subplot(1,2,1)
    plt.imshow(rgb_img)
    plt.title(
        f"True: {class_names[label]}"
    )
    plt.axis("off")

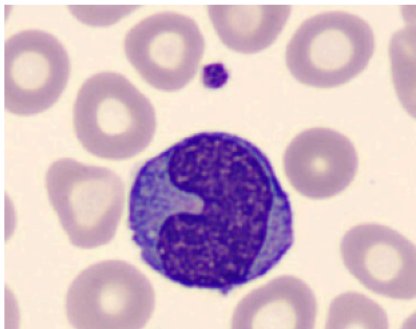
    plt.subplot(1,2,2)
    plt.imshow(cam_image)
    plt.title(
        f"Pred: {class_names[pred]}"
    )
    plt.axis("off")

    plt.show()

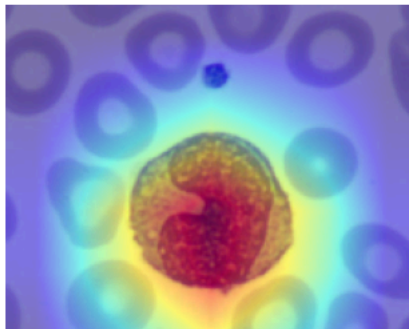
    print(
        f"Confidence: {confidence:.4f}"
    )

```


True: monocyte



Pred: monocyte



```
import torch.nn.functional as F
import numpy as np

def mc_dropout_predict(
    model,
    image,
    T=30
):
    enable_dropout(model)

    predictions = []

    with torch.no_grad():
        for _ in range(T):
            output = model(image)

            probs = F.softmax(
                output,
                dim=1
            )

            predictions.append(
                probs.cpu().numpy()
            )

    predictions = np.array(
        predictions
    )

    mean_probs = predictions.mean(
        axis=0
    )

    std_probs = predictions.std(
        axis=0
    )

    return mean_probs, std_probs
```

```
plt.figure(figsize=(10,4))

plt.plot(
    history["train_loss"],
    label="Train Loss"
)

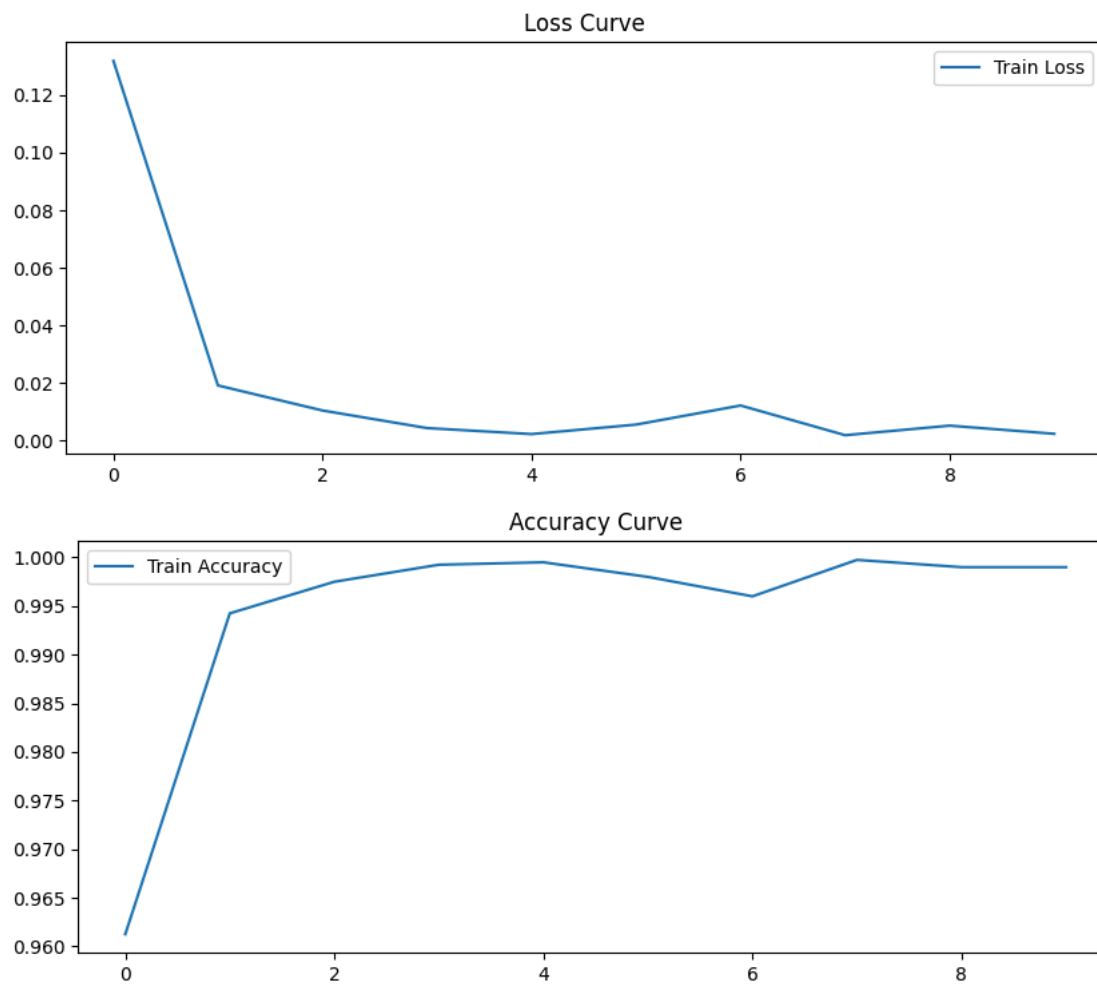
plt.legend()
plt.title("Loss Curve")
plt.show()

plt.figure(figsize=(10,4))

plt.plot(
    history["train_acc"],
    label="Train Accuracy"
)

plt.legend()
plt.title("Accuracy Curve")
plt.show()
```





```

image, label = test_dataset[0]

input_tensor = image.unsqueeze(0).to(device)

mean_probs, std_probs = mc_dropout_predict(
    model,
    input_tensor,
    T=30
)

print("Mean probabilities:")
print(mean_probs)

print()

print("Uncertainty:")
print(std_probs)

print()

print(
    "Max uncertainty:",
    std_probs.max()
)
uncertainties = []
confidences = []

for idx in range(len(test_dataset)):

    image, label = test_dataset[idx]

    input_tensor = (
        image.unsqueeze(0)
        .to(device)
    )

    mean_probs, std_probs = mc_dropout_predict(
        model,
        input_tensor,
        T=30
    )

```

```
confidences.append(  
    mean_probs.max()  
)  
  
uncertainties.append(  
    std_probs.max()  
)
```

Mean probabilities:
[[7.6559172e-06 3.5321204e-06 9.9997872e-01 7.7149352e-06 2.4746951e-06]]

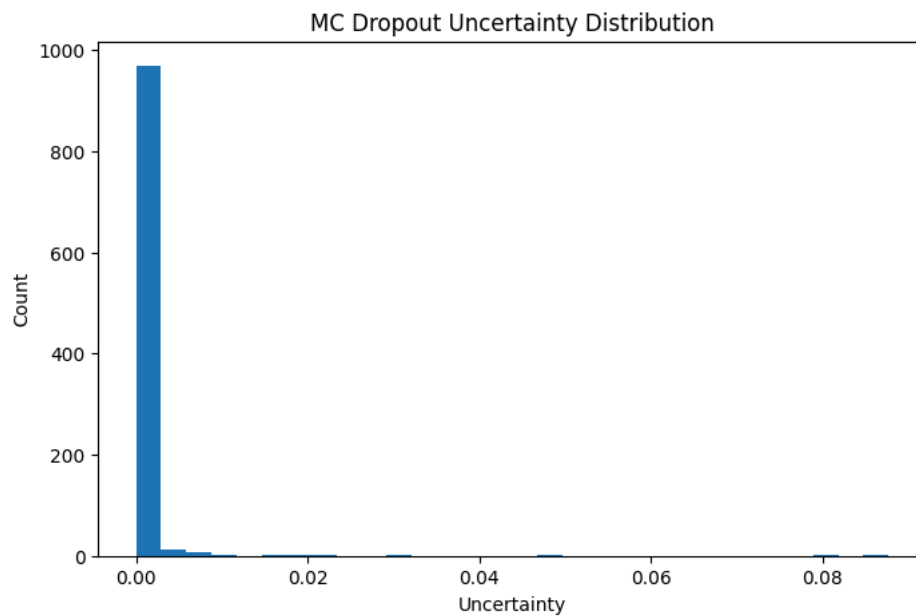
Uncertainty:
[[6.8994923e-06 2.7336187e-06 1.6106987e-05 7.0748483e-06 2.7485100e-06]]

Max uncertainty: 1.6106987e-05

```
print(  
    "Mean Uncertainty:",  
    np.mean(uncertainties)  
)  
  
print(  
    "Std Uncertainty:",  
    np.std(uncertainties)  
)  
  
print(  
    "Max Uncertainty:",  
    np.max(uncertainties)  
)  
  
print(  
    "Min Uncertainty:",  
    np.min(uncertainties)  
)
```

Mean Uncertainty: 0.00071307673
Std Uncertainty: 0.0052670976
Max Uncertainty: 0.08769219
Min Uncertainty: 2.6298193e-07

```
plt.figure(figsize=(8,5))  
  
plt.hist(  
    uncertainties,  
    bins=30  
)  
  
plt.xlabel(  
    "Uncertainty"  
)  
  
plt.ylabel(  
    "Count"  
)  
  
plt.title(  
    "MC Dropout Uncertainty Distribution"  
)  
  
plt.show()
```



```

model.eval()

confidences = []
correctness = []

with torch.no_grad():
    for images, labels in test_loader:

        images = images.to(device)

        outputs = model(images)

        probs = F.softmax(
            outputs,
            dim=1
        )

        conf, pred = probs.max(1)

        confidences.extend(
            conf.cpu().numpy()
        )

        correctness.extend(
            (
                pred.cpu()
                ==
                labels
            ).cpu().numpy()
        )

```

```

from sklearn.calibration import calibration_curve

prob_true, prob_pred = calibration_curve(
    correctness,
    confidences,
    n_bins=10
)

plt.figure(figsize=(6,6))

plt.plot(
    prob_pred,
    prob_true,
    marker="o"
)

plt.plot(
    [0,1],
    [0,1],
    "--"
)

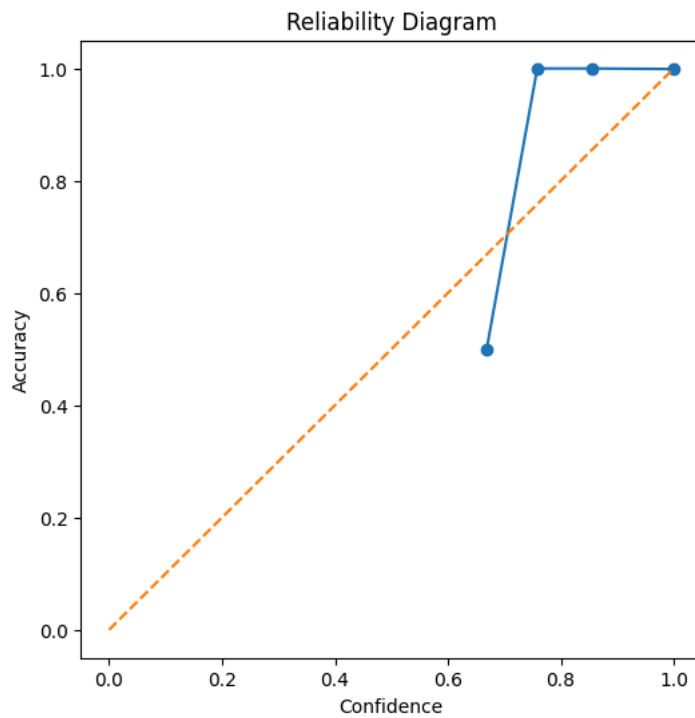
plt.xlabel("Confidence")

```

```
plt.ylabel("Accuracy")

plt.title(
    "Reliability Diagram"
)

plt.show()
```



```
def compute_ece(
    confidences,
    correctness,
    n_bins=10
):
    bins = np.linspace(
        0,
        1,
        n_bins+1
    )

    ece = 0

    for i in range(n_bins):
        mask = (
            (np.array(confidences) > bins[i])
            &
            (np.array(confidences) <= bins[i+1])
        )

        if mask.sum() == 0:
            continue

        acc = np.mean(
            np.array(correctness)[mask]
        )

        conf = np.mean(
            np.array(confidences)[mask]
        )

        ece += (
            mask.mean()
            *
            abs(acc-conf)
        )

    return ece
```

```

ece = compute_ece(
    confidences,
    correctness
)

print(
    f"ECE = {ece:.4f}"
)

```

```
ECE = 0.0012
```

```

def fgsm_attack(
    image,
    epsilon,
    data_grad
):

    sign_data_grad = data_grad.sign()

    perturbed_image = (
        image
        +
        epsilon * sign_data_grad
    )

    return perturbed_image

```

```

epsilons = [
    0,
    0.005,
    0.01,
    0.02,
    0.05
]

fgsm_accs = []

```

```

for epsilon in epsilons:

    correct = 0

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        images.requires_grad = True

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        model.zero_grad()

        loss.backward()

        data_grad = images.grad.data

        perturbed_data = fgsm_attack(
            images,
            epsilon,
            data_grad
        )

        outputs = model(
            perturbed_data
        )

        preds = outputs.argmax(1)

        correct += (
            preds == labels
        ).sum().item()

    acc = correct / len(test_dataset)

```

```
fgsm_accs.append(acc)

print(
    f"Epsilon={epsilon} "
    f"Accuracy={acc:.4f}"
)
```

```
Epsilon=0 Accuracy=0.9980
Epsilon=0.005 Accuracy=0.9520
Epsilon=0.01 Accuracy=0.8770
Epsilon=0.02 Accuracy=0.7260
Epsilon=0.05 Accuracy=0.5020
```

```
plt.figure(figsize=(8,5))

plt.plot(
    epsilons,
    fgsm_accs,
    marker="o"
)

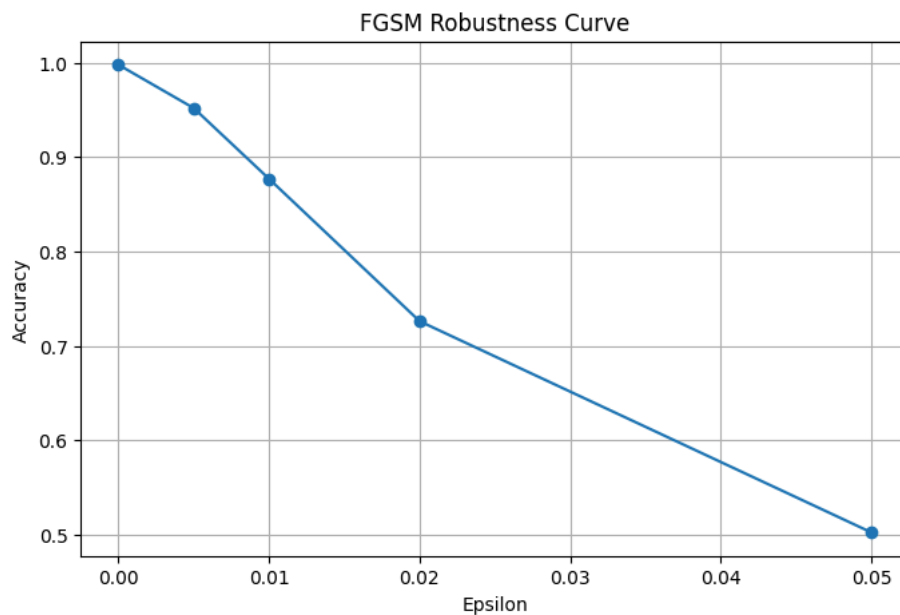
plt.xlabel(
    "Epsilon"
)

plt.ylabel(
    "Accuracy"
)

plt.title(
    "FGSM Robustness Curve"
)

plt.grid()

plt.show()
```



```
import matplotlib.pyplot as plt
import torch.nn.functional as F
import numpy as np

epsilon = 0.06

image, label = test_dataset[0]

input_tensor = image.unsqueeze(0).to(device)
target = torch.tensor([label]).to(device)

input_tensor.requires_grad = True

output = model(input_tensor)
loss = criterion(output, target)

model.zero_grad()
loss.backward()
```

```

data_grad = input_tensor.grad.data

perturbed = fgsm_attack(
    input_tensor,
    epsilon,
    data_grad
)

with torch.no_grad():

    output_before = model(input_tensor)
    prob_before = F.softmax(output_before, dim=1)

    pred_before = prob_before.argmax(1).item()
    conf_before = prob_before.max().item()

    output_after = model(perturbed)
    prob_after = F.softmax(output_after, dim=1)

    pred_after = prob_after.argmax(1).item()
    conf_after = prob_after.max().item()

orig = image.permute(1,2,0).cpu().numpy()
orig = (orig - orig.min()) / (orig.max() - orig.min())

adv = perturbed.squeeze().detach().cpu().permute(1,2,0).numpy()
adv = (adv - adv.min()) / (adv.max() - adv.min())

noise = adv - orig

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(orig)
plt.title(
    f"Original\n{class_names[pred_before]}\n{conf_before:.4f}"
)
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(
    np.abs(noise),
    cmap="hot"
)
plt.colorbar()
plt.title("FGSM Noise")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(adv)
plt.title(
    f"FGSM  $\epsilon$ ={epsilon}\n{class_names[pred_after]}\n{conf_after:.4f}"
)
plt.axis("off")

plt.show()

print("True Label :", class_names[label])
print("Before FGSM:", class_names[pred_before], conf_before)
print("After FGSM :", class_names[pred_after], conf_after)

```